

Day 52 -- Line-by-Line Syntax Explanation

day52_vgg_style_stacked_convs.py, segment by segment | MD Mutasim Billah

1. Shared Helpers Reused From Days 49-51

conv2d_forward_mc/conv2d_backward_mc, maxpool_forward/backward, relu/drelu, softmax, cross_entropy_loss, he_scale, iterate_minibatches, make_junction_dataset_rgb, and train_with_history are reused byte-for-byte from Day 49/50, now with an inline comment on every line per this lesson's new convention -- their logic is unchanged, so see Day 49's syntax guide for their original line-by-line walkthrough. This guide focuses on what's genuinely new today.

2. LargeKernelConvNet -- the Control Architecture

`__init__`: two blocks, each ONE 5x5 conv

```
self.Wc1 = he_scale(rng, (C, 3, 5, 5)) # block 1: 3 input channels -> C output channels, 5x5 kernel
self.Wc2 = he_scale(rng, (C, C, 5, 5)) # block 2: C -> C, 5x5 kernel
```

Both conv weight tensors use He initialization (`he_scale`), matching Days 49-51's convention: scale by $\sqrt{2 / \text{fan_in}}$ so variance is preserved through a ReLU non-linearity. This network is the "one large kernel" side of today's comparison -- 2 conv layers total.

3. VGGStyleConvNet -- the New Stacked-Conv Architecture

`__init__`: two blocks, each TWO stacked 3x3 convs

```
self.Wc1a = he_scale(rng, (C, 3, 3, 3)) # block 1, conv a: 3 -> C, 3x3 kernel
self.Wc1b = he_scale(rng, (C, C, 3, 3)) # block 1, conv b: C -> C, 3x3 kernel
self.Wc2a = he_scale(rng, (C, C, 3, 3)) # block 2, conv a: C -> C, 3x3 kernel
self.Wc2b = he_scale(rng, (C, C, 3, 3)) # block 2, conv b: C -> C, 3x3 kernel
```

Four conv weight tensors instead of two -- this is what makes the network twice as deep in its conv path. Note the channel width stays fixed at C throughout (3->C->C->C->C), which is what makes the $18C^2$ -vs- $25C^2$ parameter comparison exact and fair: both networks use the identical C, just arranged into different kernel shapes.

`forward`: two convs, ReLU after each, ONE pool per block

```
Zc1a = conv2d_forward_mc(X, self.Wc1a) # first 3x3 conv of block 1
Ac1a = relu(Zc1a) # non-linearity #1
Zc1b = conv2d_forward_mc(Ac1a, self.Wc1b) # second 3x3 conv of block 1, fed the FIRST conv's output
Ac1b = relu(Zc1b) # non-linearity #2
P1, argmax1 = maxpool_forward(Ac1b) # pool AFTER both convs, not after each one
```

This is the one detail that must be exactly right for the receptive-field-equivalence claim to hold: pooling only once, after both 3x3 convs, is what makes VGGStyleConvNet's post-block spatial size match LargeKernelConvNet's exactly. Pooling after conv1a (before conv1b) would change the comparison entirely. Both `Zc1a` and `Zc1b` are cached on self (`self.Zc1a`, `self.Zc1b`, ...) specifically so `measure_dead_fraction()` can inspect their sign statistics after training.

backward: two conv2d_backward_mc calls chained through the ReLU derivative

```
dAc1b = maxpool_backward(dP1, argmax1, Ac1b.shape) # undo the pool
dZc1b = dAc1b * drelu(Zc1b) # undo ReLU #2
dAc1a, dWc1b = conv2d_backward_mc(dZc1b, Ac1a, self.Wc1b) # undo conv b, get grad for conv
a's output too
dZc1a = dAc1a * drelu(Zc1a) # undo ReLU #1
_, dWc1a = conv2d_backward_mc(dZc1a, X, self.Wc1a) # undo conv a, get grad for its own
weights
```

This is the new backward chain that Days 49-51 never needed: the gradient has to pass through TWO conv layers and TWO ReLUs within a single block, not one of each. Getting the order and the drelu(Zc1a) vs drelu(Zc1b) placement right is exactly what the gradient check in Section 2 of the .py script verifies before this network is ever trusted to train.

4. measure_dead_fraction() -- Diagnosing the Training Failure

measuring what fraction of each layer is dead under ReLU

```
def measure_dead_fraction(net): # takes a trained VGGStyleConvNet
    return dict(Zc1a=(net.Zc1a <= 0).mean(), # fraction of block-1 conv-a pre-activations <= 0
                Zc1b=(net.Zc1b <= 0).mean(), # same, conv b of block 1
                Zc2a=(net.Zc2a <= 0).mean(), # same, conv a of block 2
                Zc2b=(net.Zc2b <= 0).mean()) # same, conv b of block 2
```

This function was added specifically to turn a suspicious result (VGGStyleConvNet not training) into a measured, diagnosed one. It reads the SAME cached pre-activation arrays the backward pass already needed (Zc1a, Zc1b, Zc2a, Zc2b), so no extra forward pass is required -- it is a pure diagnostic on state the network already computed.

5. demo_train_and_compare() -- the Two-Stage Comparison

stage 3a: both networks at the inherited lr=0.15

```
hist_large_hi = train_with_history(large_hi, Xtr, ytr, Xte, yte,
    lr=0.15, epochs=150, batch_size=50, seed=42)
hist_vgg_hi = train_with_history(vgg_hi, Xtr, ytr, Xte, yte,
    lr=0.15, epochs=150, batch_size=50, seed=42)
```

Both networks are trained with the exact same call signature and the exact same lr=0.15 that worked for Days 50-51 -- deliberately, so the ONLY variable that differs between this run and Day 51's is kernel shape. This is what makes VGGStyleConvNet's failure attributable to depth rather than to some other confound like a different dataset seed or batch size.

stage 3b: both networks retrained at the recalibrated lr=0.03

```
hist_large_lo = train_with_history(large_lo, Xtr, ytr, Xte, yte,
    lr=0.03, epochs=150, batch_size=50, seed=42)
hist_vgg_lo = train_with_history(vgg_lo, Xtr, ytr, Xte, yte,
    lr=0.03, epochs=150, batch_size=50, seed=42)
winner = "LargeKernelConvNet" if te_large_lo > te_vgg_lo else "VGGStyleConvNet"
```

lr=0.03 came from the calibration sweep (measuring dead-unit fraction at lr=0.05/0.03/0.01) printed just before this call in the script. Both networks are retrained FROM SCRATCH (fresh he_scale init, same seed) at the new rate -- this is a fair rematch, not a continuation of the failed run. The `winner` line computes which network actually generalized better from the real numbers, rather than assuming the answer in advance.

6. note_on_pytorch() -- Mapping to a Real Framework

the from-scratch block collapses to a few nn.Module lines

```
self.conv_a = nn.Conv2d(c_in, c_out, kernel_size=3) # replaces Wc1a +  
conv2d_forward_mc/backward_mc  
self.conv_b = nn.Conv2d(c_out, c_out, kernel_size=3) # replaces Wc1b + its own  
forward/backward calls  
self.pool = nn.MaxPool2d(2, 2) # replaces maxpool_forward/backward
```

PyTorch's autograd computes the two-conv, two-ReLU backward chain automatically the moment forward() is called with requires_grad=True tensors -- the manual chaining of conv2d_backward_mc calls done by hand in this lesson's backward() method is exactly what autograd is doing under the hood, just without needing it spelled out.